


```
List<Point> X = ArrayList<>();  
Disk d = new Disk();  
int c;
```

Simple classes II.

```

c := 0;
for (Point e : x) {
  if (e in d) {
    c = c + 1;
  }
}
    
```

How many of the given points fall within a specific circular disc?

Specification:

$S = (x: \text{Point}^*, d: \text{Disc}, c: \mathbb{N})$

$Q = (x=x' \wedge d=d')$

$R = (R \wedge c = \sum_{e \in k} 1)$

Counting

$\text{enor}(\text{Point}) \sim \text{enumerating Point}^*$

$\text{cond}(i) \sim e \in k$

Algorithm:

$c := 0$	
$e \in x$	$e: \text{Point}$
$e \in d$	
$c := c + 1$	—

Note: If x is a $[1..n]$ indexed array, we could list the elements of the array using a for loop with $i=1..n$; instead of the e variable, we would use $x[i]$. That would be of course a worse solution.

Type of Point and Disc (circle), representing the Discs with their centers and radii, and the points with their coordinates.

Type definitions:

Disc

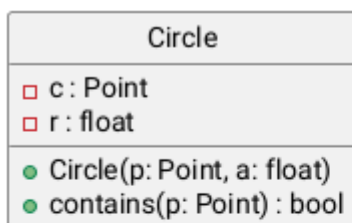
discs	$l := p \in k \quad (d: \text{Disc}, p: \text{Point}, l:)$
$c : \text{Point}$ $r : \mathbb{R}$ Inv: $r \geq 0$	$l := \overline{c}, p \leq r$

Point

points	$d := \overline{q}, p \quad (p, q : \text{Point}, d: \mathbb{R})$
$x, y : \mathbb{R}$	$d := \sqrt{(p.x - q.x)^2 + (p.y - q.y)^2}$

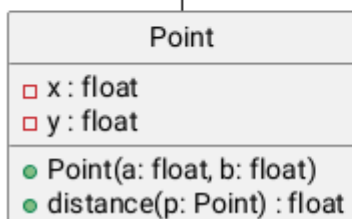
Note: Many times we do go top-down during design, but bottom-up when coding in OOP. Bottom-up can also work for design, or the combination of the two.

Classes:



Circle(p, a):
if $a < 0$ then error endif
 $c, r := p, a$

contains(p):
return $c.\text{distance}(p) \leq r$



Point(a, b):
 $x, y := a, b$

distance(p):
return $\text{sqrt}((x - p.x)^2 + (y - p.y)^2)$

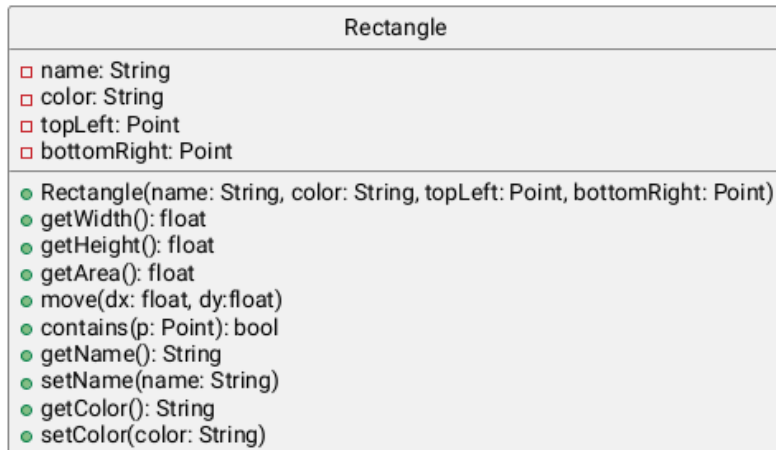
Note: The distance between two points and the distance of one point from another are not different concepts, but as methods they have different signatures. Here the second interpretation appears: call the `distance()` method of a specific point (`c`) with another point (`p`) to calculate the distance between the two points: `c.distance(p)`. In the following examples, however, we encounter operations on two objects of equal weight, which are therefore introduced as class-level methods. For the distance example, it would be `Point.distance(c, p)`.

Exercise: Rectangle

A Rectangle has a name, a color, and is defined by two Points, representing its top-left and bottom-right corners. Each Point has an x and y coordinate and provides methods to retrieve and modify them. Since a Rectangle cannot exist without its defining Points, it maintains a Composition relationship with them.

The Rectangle calculates its width and height based on the difference between the x and y coordinates of these points. The rectangle can also compute its area by multiplying its width and height.

The Rectangle can be moved by shifting both of its points by a given offset (dx, dy). Additionally, it can check whether a given Point lies inside its boundaries by comparing the point's coordinates with its edges. Both the Point and Rectangle classes should have a constructor, getters, setters, and appropriate methods to support these functionalities.



```

Rectangle(name, color, topLeft, bottomRight):
    this.name = name
    this.color = color
    this.topLeft = topLeft
    this.bottomRight = bottomRight

getWidth():
    return abs(bottomRight.getX() - topLeft.getX())

getHeight():
    return abs(topLeft.getY() - bottomRight.getY())

getArea():
    return getWidth()*getHeight()

move(dx,dy):
    topLeft.setX(topLeft.getX() + dx)
    topLeft.setY(topLeft.getY() + dy)
    bottomRight.setX(bottomRight.getX() + dx)
    bottomRight.setY(bottomRight.getY() + dy)

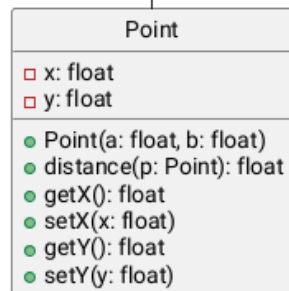
contains(p):
    return p.getX() ≥ topLeft.getX() ∧
           p.getX() ≤ bottomRight.getX() ∧
           p.getY() ≤ topLeft.getY() ∧
           p.getY() ≥ bottomRight.getY()

getName():
    return this.name

setName(name):
    this.name = name

getColor():
    return this.color

setColor(color):
    this.color = color
  
```



```

Point(a, b):
    x, y := a, b

distance(p):
    return sqrt((x - p.x)^2 + (y - p.y)^2)

getX():
    return this.x

setX(x):
    this.x = x

getY():
    return this.y

setY(y):
    this.y = y
  
```

Adding second-degree polynomials, multiplying by a scalar, and calculating the value by substitution.

Type definition:

Second order polynomials	$h := \text{value}(p, x) \quad (p : \text{Polynomial}, x : \mathbb{R}, h : \mathbb{R})$
	$p3 := p1 \pm p2 \quad (p1, p2, p3 : \text{Polynomial})$
	$p2 := p1 \cdot s \quad (p1, p2 : \text{Polynomial}, s : \mathbb{R})$
$a, b, c : \mathbb{R}$ // $ax^2 + bx + c$	$h := a \cdot x^2 + b \cdot x + c$
	$p3.a, p3.b, p3.c :=$ $p1.a \pm p2.a, p1.b \pm p2.b, p1.c \pm p2.c$
	$p2.a, p2.b, p2.c :=$ $p1.a \cdot s, p1.b \cdot s, p1.c \cdot s$

Class:

Polinomial	
□ a : float □ b : float □ c : float ● Polinomial(x : float, y : float, z : float) ● value(x : float) : float ● add(p : Polinomial, q : Polinomial) : Polinomial ● sub(p : Polinomial, q : Polinomial) : Polinomial ● mul(p : Polinomial, s : float) : Polinomial	<pre> Polinomial(x, y, z : float) : a, b, c := x, y, z value(x : float) : return a * x^2 + b * x + c add(p : Polinomial, q : Polinomial) : return Polinomial(p.a + q.a, p.b + q.b, p.c + q.c) sub(p : Polinomial, q : Polinomial) : return Polinomial(p.a - q.a, p.b - q.b, p.c - q.c) mul(p : Polinomial, s : float) : return Polinomial(p.a * s, p.b * s, p.c * s) </pre>

Exercise for using the type:

Given an array containing real numbers, and a quadratic polynomial, which element of the array gives the maximum value when substituted into the quadratic polynomial?

Specification:

$S = (p : \text{Polynomial}, v : \mathbb{R}^n, \text{ind} : \mathbb{Z}, \text{max} : \mathbb{R})$

$Q = (p = p' \wedge v = v')$

$R = (Q \wedge (\text{max}, \text{ind}) = \text{MAX}_{i=1..n} \text{value}(p, v[i]))$

Algorithm:

max, ind, := value(p, v[1]),1	
i = 2.. n	
i:ℤ	
max < value(p, v[i])	
max, ind := value(p, v[i]), i	SKIP

Exercise: Warriors

Warriors are fighting each other. They have a name and some hitpoints (an integer). Each warrior has a sword and a shield.

Warriors use swords to attack each other. The sword does a combo attack: each consecutive hit increases its damage until maxCombo hits, which depends on the sword. The warrior can choose to do a shield bash, which resets the combo counter, but increases damage by 25 times the block probability of the shield.

The shield can completely block attacks with a block probability between 0 and 1. It also becomes weaker as it blocks more and more attacks: each time it blocks an attack its durability decreases, which also decreases its block probability. The warrior can choose to repair their shield instead of attacking, which restores its durability.

The main function should instantiate two (or more) warriors and simulate their combat (call their attack, bash, and repair methods). You can use random() to generate random numbers.